

MC - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4224 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 3914 values"
## [1] "310 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	509	32	124	NA	NA	665
2	61	NA	381	90	98	630
3	283	NA	138	181	15	617
4	261	NA	210	59	35	565
5	207	NA	194	62	36	499
6	192	NA	126	65	20	403
7	164	NA	90	24	16	294
8	81	NA	50	25	4	160
9	72	NA	2	NA	7	81

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7654135	0.0481203	0.1864662	NA	NA	665
2	0.0968254	NA	0.6047619	0.1428571	0.1555556	630
3	0.4586710	NA	0.2236629	0.2933549	0.0243112	617
4	0.4619469	NA	0.3716814	0.1044248	0.0619469	565
5	0.4148297	NA	0.3887776	0.1242485	0.0721443	499
6	0.4764268	NA	0.3126551	0.1612903	0.0496278	403
7	0.5578231	NA	0.3061224	0.0816327	0.0544218	294
8	0.5062500	NA	0.3125000	0.1562500	0.0250000	160
9	0.8888889	NA	0.0246914	NA	0.0864198	81

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position
```

```
syll_comp_plot <- ggplot(syll_comp_plot_data,
  aes(x=pos,y=percent,group=syll_component,
    color=syll_component,
    linetype = syll_component,
    shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_pos_of_syll_complexities_in_stimuli.png"),plot=s
```

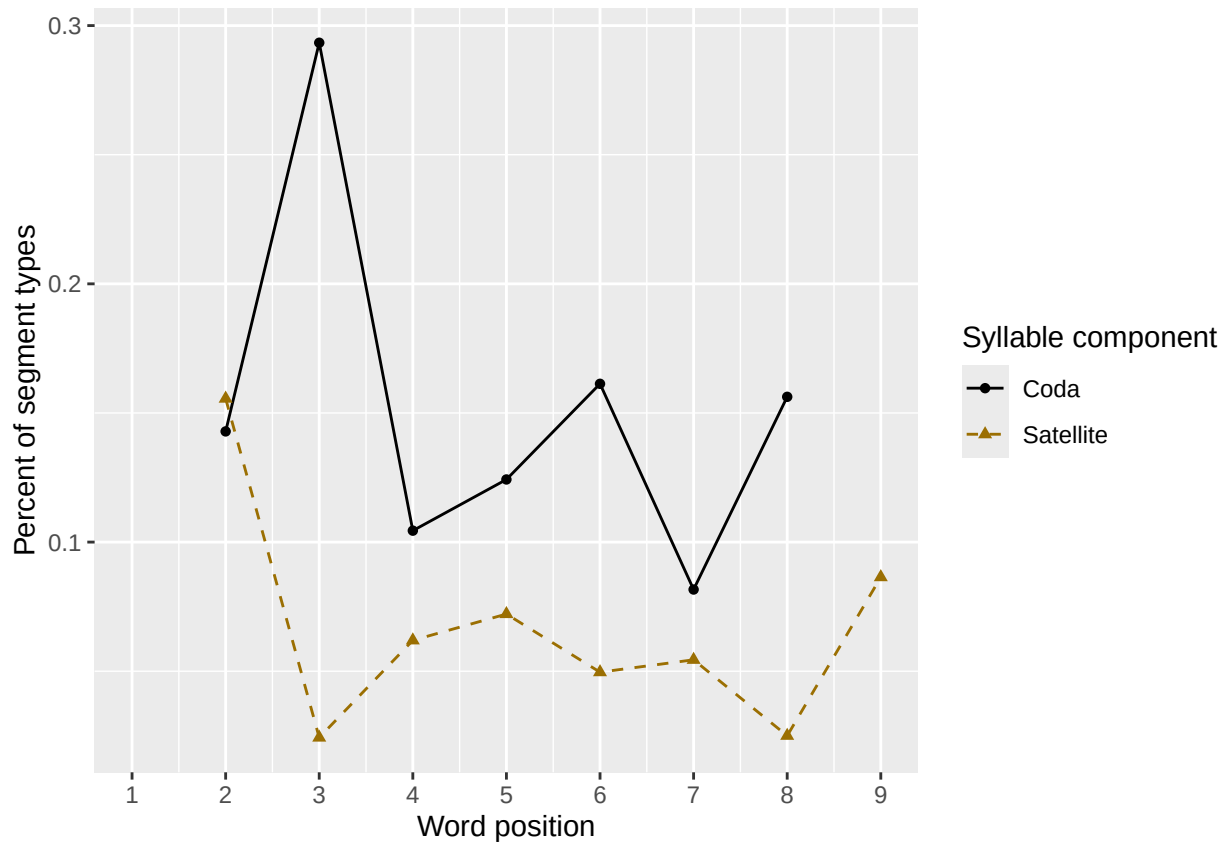
```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```



```
# len/pos table
```

```
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
```

```
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
```

```
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.764 0.833 0.648 NA      NA      NA      NA      NA      NA
## 2      5 0.735 0.821 0.736 0.584 NA      NA      NA      NA      NA
## 3      6 0.615 0.814 0.683 0.74  0.6  NA      NA      NA      NA
## 4      7 0.752 0.818 0.711 0.693 0.794 0.619 NA      NA      NA
## 5      8 0.795 0.822 0.719 0.708 0.625 0.75  0.657 NA      NA
## 6      9 0.742 0.793 0.724 0.701 0.675 0.639 0.759 0.627 NA
## 7     10 0.735 0.862 0.695 0.688 0.716 0.557 0.68  0.779 0.580

```

```

# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))
pos_len_N_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    55    54    54    NA    NA    NA    NA    NA    NA
## 2      5    83    78    72    77    NA    NA    NA    NA    NA
## 3      6   104   102   101   100   100    NA    NA    NA    NA
## 4      7   105    99    97   101   102   105    NA    NA    NA
## 5      8   146   135   135   130   136   136   140    NA    NA
## 6      9    89    82    76    77    80    83    79    83    NA
## 7     10    83    80    82    80    81    79    75    77    81

```

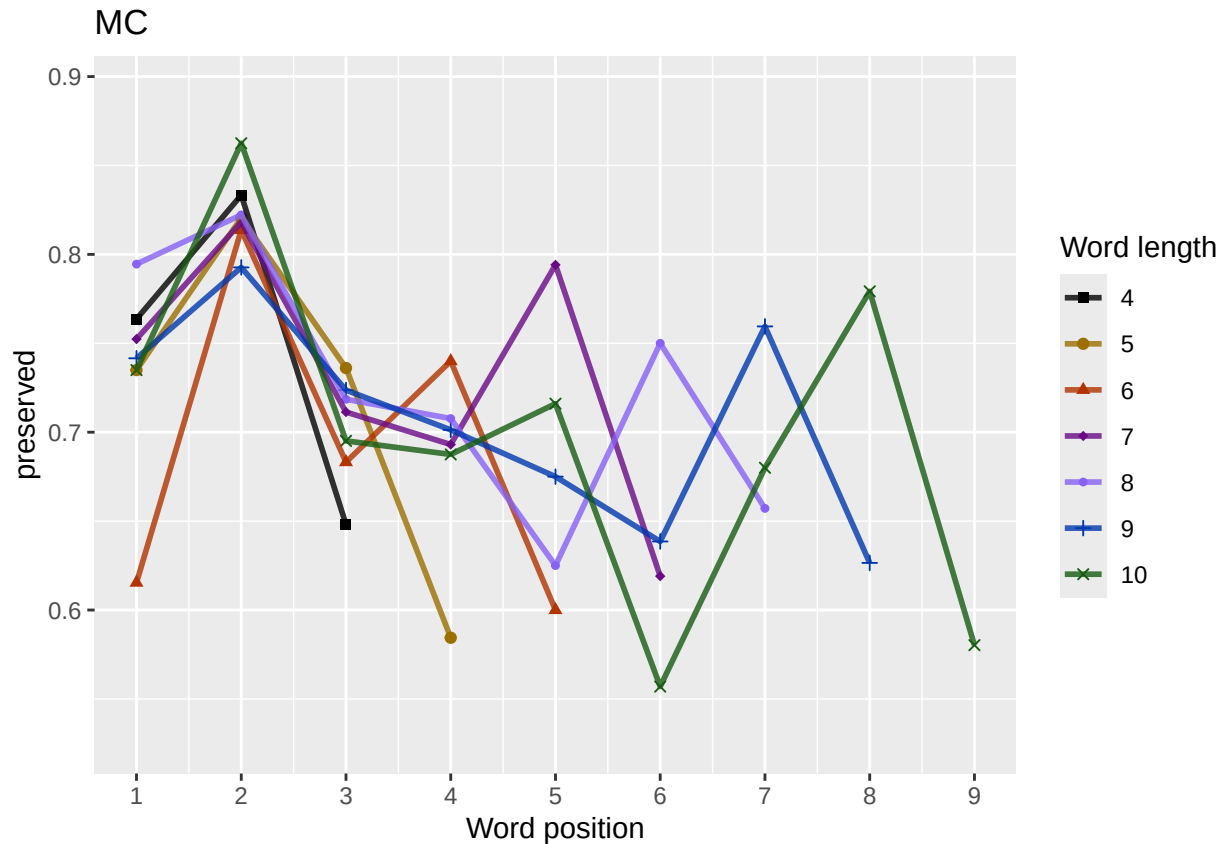
```

obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position

LPModelEquations<-c("preserved ~ 1",
                    "preserved ~ stimlen",
                    "preserved ~ pos",
                    "preserved ~ stimlen + pos",
                    "preserved ~ stimlen*pos",
                    "preserved ~ I(pos^2)+pos",
                    "preserved ~ stimlen + I(pos^2) + pos",
                    "preserved ~ stimlen * (I(pos^2) + pos)"
)

LPRes<-TestModels(LPModelEquations,PosDat)
```

```
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
```

```

##
## Coefficients:
## (Intercept)          pos
##      1.25078      -0.08522
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3912 Residual
## Null Deviance:      4683
## Residual Deviance: 4655  AIC: 4659
## log likelihood:  -2327.574
## Nagelkerke R2:  0.01006489
## % pres/err predicted correctly:  -1585.017
## % of predictable range [ (model-null)/(1-null) ]:  0.007067415
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      1.04178      0.03226     -0.09544
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3911 Residual
## Null Deviance:      4683
## Residual Deviance: 4653  AIC: 4659
## log likelihood:  -2326.598
## Nagelkerke R2:  0.01077426
## % pres/err predicted correctly:  -1584.249
## % of predictable range [ (model-null)/(1-null) ]:  0.007548574
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      1.351626      0.007005     -0.146799
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3911 Residual
## Null Deviance:      4683
## Residual Deviance: 4654  AIC: 4660
## log likelihood:  -2327.101
## Nagelkerke R2:  0.01040858
## % pres/err predicted correctly:  -1584.659
## % of predictable range [ (model-null)/(1-null) ]:  0.007291756
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos

```

```

##      1.139562      0.029311      0.005467      -0.142553
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3910 Residual
## Null Deviance:      4683
## Residual Deviance: 4653 AIC: 4661
## log likelihood: -2326.319
## Nagelkerke R2: 0.01097736
## % pres/err predicted correctly: -1584.041
## % of predictable range [ (model-null)/(1-null) ]: 0.007678912
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos      stimlen:pos
##      1.247056      0.007250     -0.161081      0.007727
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3910 Residual
## Null Deviance:      4683
## Residual Deviance: 4653 AIC: 4661
## log likelihood: -2326.355
## Nagelkerke R2: 0.01095096
## % pres/err predicted correctly: -1584.035
## % of predictable range [ (model-null)/(1-null) ]: 0.007682574
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      stimlen:I(pos^2)      stimlen:I(pos^2)
##      0.980732      0.044787     -0.019952      0.007793      0.002651      -0.002651
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3908 Residual
## Null Deviance:      4683
## Residual Deviance: 4652 AIC: 4664
## log likelihood: -2326.145
## Nagelkerke R2: 0.01110358
## % pres/err predicted correctly: -1583.908
## % of predictable range [ (model-null)/(1-null) ]: 0.007762152
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      0.9166
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3913 Residual

```



```
## Null Deviance:          4683
## Residual Deviance: 4683 AIC: 4685
## log likelihood: -2341.365
## Nagelkerke R2: -3.182426e-16
## % pres/err predicted correctly: -1596.306
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      1.04811      -0.01702
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3912 Residual
## Null Deviance:          4683
## Residual Deviance: 4682 AIC: 4686
## log likelihood: -2341.034
## Nagelkerke R2: 0.000242303
## % pres/err predicted correctly: -1596.036
## % of predictable range [ (model-null)/(1-null) ]: 0.0001692916
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                             "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ pos	4659.148	0.000000	0.000000	0.0279698	0.1006492	507836	NA	-	NA	NA	NA
								0.0852246			
preserved ~ stimlen + pos	4659.196	0.0486108	0.9759876	0.2729808	0.1077430	417826	0.0322601	-	NA	NA	NA
								0.0954368			
preserved ~ I(pos^2) + pos	4660.202	0.0546574	0.5901794	0.1650720	0.1040863	516260	NA	-	NA	0.0070055	NA
								0.1467994			
preserved ~ stimlen + I(pos^2) + pos	4660.637	0.4897135	0.4748023	0.3132800	0.3010977	4139562	0.0293110	-	NA	0.0054673	NA
								0.1425526			

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * pos	4660.710	0.562363	36457864	0.6128063	0.8010951	0.2470566	0.0072499	-	0.0077273	NA	NA
								0.1610809			
preserved ~ stimlen * (I(pos^2) + pos)	4664.290	0.142341	1807644	0.6002138	0.8011108	0.9807316	0.0447866	0.0077930	-	-	0.0026508
									0.0153690	0.0199517	
preserved ~ 1	4684.732	5.582968	5000000	0.2800000	0.0800000	0.0091664	84 NA	NA	NA	NA	NA
preserved ~ stimlen	4686.062	6.921212	12000000	0.4000000	0.0400024	230481056	-	NA	NA	NA	NA
								0.0170186			

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ pos"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)          pos
```

```
##      1.25078      -0.08522
```

```
##
```

```
## Degrees of Freedom: 3913 Total (i.e. Null); 3912 Residual
```

```
## Null Deviance: 4683
```

```
## Residual Deviance: 4655 AIC: 4659
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],  
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
```

```
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups:   stimlen [7]
```

```
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`  
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1      4 0.762 0.747 0.730 NA      NA      NA      NA      NA      NA
```

```
## 2      5 0.762 0.747 0.730 0.713 NA      NA      NA      NA      NA
```

```
## 3      6 0.762 0.747 0.730 0.713 0.695 NA      NA      NA      NA
```

```
## 4      7 0.762 0.747 0.730 0.713 0.695 0.677 NA      NA      NA
```

```
## 5      8 0.762 0.747 0.730 0.713 0.695 0.677 0.658 NA      NA
```

```
## 6      9 0.762 0.747 0.730 0.713 0.695 0.677 0.658 0.639 NA
```

```
## 7     10 0.762 0.747 0.730 0.713 0.695 0.677 0.658 0.639 0.619
```

```

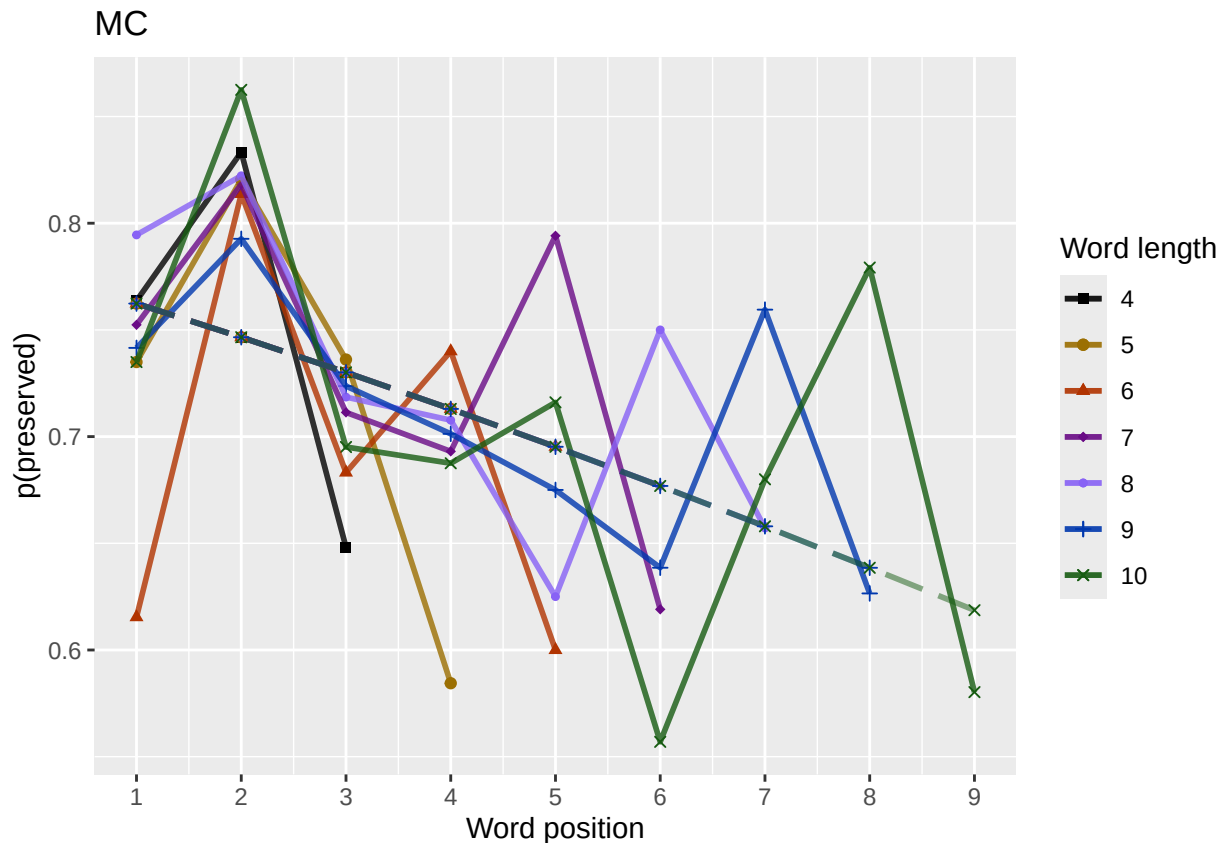
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen,color=stimlen))
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen,color=stimlen)) + ggtitle(paste0("Patient",patient))

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_percent_preserved_by_length_pos_wfit.png"),plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

```

```

# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      33   681

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 33 / 681 = 4.85 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##   1.20806    -0.05467
##
## Degrees of Freedom: 3826 Total (i.e. Null); 3825 Residual
## Null Deviance: 4460
## Residual Deviance: 4449 AIC: 4453
## log likelihood: -2224.55
## Nagelkerke R2: 0.004019653
## % pres/err predicted correctly: -1501.315
## % of predictable range [ (model-null)/(1-null) ]: 0.002773662

```

```

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      1.03863      0.02606     -0.06283
##
## Degrees of Freedom: 3826 Total (i.e. Null); 3824 Residual
## Null Deviance:      4460
## Residual Deviance: 4448 AIC: 4454
## log likelihood: -2223.934
## Nagelkerke R2: 0.004486196
## % pres/err predicted correctly: -1500.845
## % of predictable range [ (model-null)/(1-null) ]: 0.003085946
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      1.289791      0.005826     -0.105383
##
## Degrees of Freedom: 3826 Total (i.e. Null); 3824 Residual
## Null Deviance:      4460
## Residual Deviance: 4448 AIC: 4454
## log likelihood: -2224.244
## Nagelkerke R2: 0.004251192
## % pres/err predicted correctly: -1501.081
## % of predictable range [ (model-null)/(1-null) ]: 0.002929042
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      1.11845      0.02366      0.00458     -0.10193
##
## Degrees of Freedom: 3826 Total (i.e. Null); 3823 Residual
## Null Deviance:      4460
## Residual Deviance: 4448 AIC: 4456
## log likelihood: -2223.75
## Nagelkerke R2: 0.004625165
## % pres/err predicted correctly: -1500.705
## % of predictable range [ (model-null)/(1-null) ]: 0.003178559
## *****
## model index: 5
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
## 1.093852      0.019329     -0.080852      0.002122
##
## Degrees of Freedom: 3826 Total (i.e. Null); 3823 Residual
## Null Deviance: 4460
## Residual Deviance: 4448 AIC: 4456
## log likelihood: -2223.916
## Nagelkerke R2: 0.004499294
## % pres/err predicted correctly: -1500.829
## % of predictable range [ (model-null)/(1-null) ]: 0.003096628
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos  stimlen:I(pos^2)      stimlen:I(pos^2)
## 1.0871239      0.0298872      0.0094431     -0.1093315     -0.0003866     -0.0003866
##
## Degrees of Freedom: 3826 Total (i.e. Null); 3821 Residual
## Null Deviance: 4460
## Residual Deviance: 4447 AIC: 4459
## log likelihood: -2223.719
## Nagelkerke R2: 0.004648482
## % pres/err predicted correctly: -1500.683
## % of predictable range [ (model-null)/(1-null) ]: 0.003193426
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 0.9977
##
## Degrees of Freedom: 3826 Total (i.e. Null); 3826 Residual
## Null Deviance: 4460
## Residual Deviance: 4460 AIC: 4462
## log likelihood: -2229.85
## Nagelkerke R2: -3.226554e-16
## % pres/err predicted correctly: -1505.494
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##

```

```

## Coefficients:
## (Intercept)      stimlen
##      1.040002    -0.005493
##
## Degrees of Freedom: 3826 Total (i.e. Null);  3825 Residual
## Null Deviance:      4460
## Residual Deviance: 4460  AIC: 4464
## log likelihood:  -2229.818
## Nagelkerke R2:   2.494468e-05
## % pres/err predicted correctly:  -1505.468
## % of predictable range [ (model-null)/(1-null) ]:  1.729951e-05
## *****

NoFragBestLPModel<-NoFrag_LPres$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPres$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPres$Model,
                                AIC=NoFrag_LPres$AIC,
                                row.names = NoFrag_LPres$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPres$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPres$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ pos	4453.100	0.000000	0.000000	0.000000	0.000000	1.040002	0.005493	-	NA	NA
								0.0546737		
preserved ~ stimlen + pos	4453.867	0.767677	0.268124	0.424351	0.390044	0.486203	0.386258	-	NA	NA
								0.0628274		
preserved ~ I(pos^2) + pos	4454.488	1.388462	0.499458	0.217853	0.400425	1.228979	0.06 NA	-	NA	0.0058258
								0.1053827		
preserved ~ stimlen + I(pos^2)	4455.502	2.400527	0.301114	0.810763	0.530046	2.321184	0.540.0236596	-	NA	0.0045797
								0.1019334		
+ pos										
preserved ~ stimlen * pos	4455.833	2.733073	0.225498	0.800911	0.720044	0.993852	0.0193289	-	0.0021222	NA
								0.0808519		
preserved ~ stimlen * (I(pos^2)	4459.439	6.338922	0.042026	0.215022	0.500464	8.508712	0.390298872	-	-	0.0094431
								0.1093315	0.0006290	0.0003866
+ pos)										
preserved ~ 1	4461.708	8.601069	0.013560	0.300484	0.760000	0.009766	0.06 NA	NA	NA	NA
preserved ~ stimlen	4463.635	10.535373	0.005155	0.500184	0.990000	2.490400	0.025	-	NA	NA
								0.0054933		

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

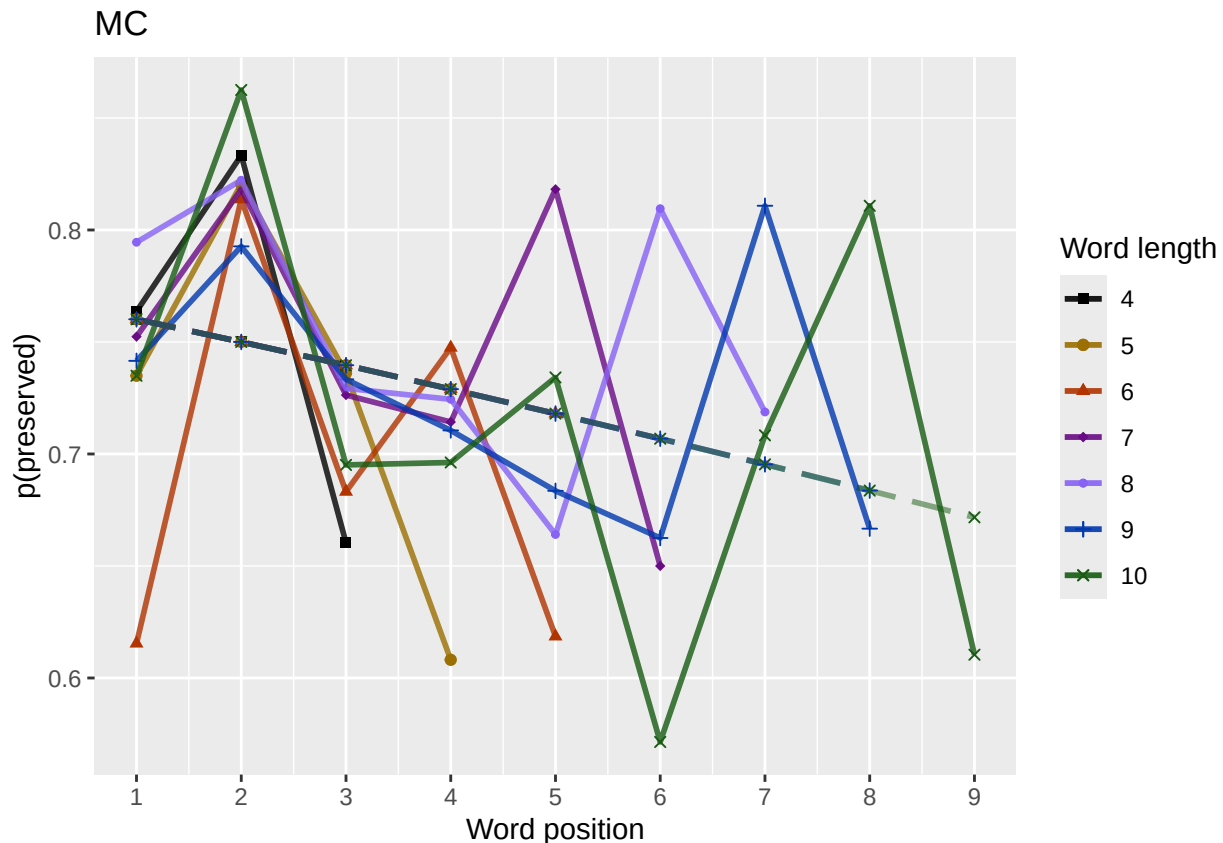
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.760 0.750 0.740 NA      NA      NA      NA      NA      NA
## 2      5 0.760 0.750 0.740 0.729 NA      NA      NA      NA      NA
## 3      6 0.760 0.750 0.740 0.729 0.718 NA      NA      NA      NA
## 4      7 0.760 0.750 0.740 0.729 0.718 0.707 NA      NA      NA
## 5      8 0.760 0.750 0.740 0.729 0.718 0.707 0.695 NA      NA
## 6      9 0.760 0.750 0.740 0.729 0.718 0.707 0.695 0.684 NA
## 7     10 0.760 0.750 0.740 0.729 0.718 0.707 0.695 0.684 0.672
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen))
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("no_fragments_percent_preserved_by_length_pos_wfit.png"))
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.53 - 0.89"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] 0
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.01796319
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
    2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentange_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentange_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small sample sizes)
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 16
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
## (Intercept)          pos
##      1.25078      -0.08522
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3912 Residual
## Null Deviance:      4683
## Residual Deviance: 4655  AIC: 4659
## log likelihood:  -2327.574
## Nagelkerke R2:  0.01006489
## % pres/err predicted correctly:  -1585.017
## % of predictable range [ (model-null)/(1-null) ]:  0.007067415
## *****
## model index:  17
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      1.04178      0.03226     -0.09544
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3911 Residual
## Null Deviance:      4683
## Residual Deviance: 4653  AIC: 4659
## log likelihood:  -2326.598
## Nagelkerke R2:  0.01077426
## % pres/err predicted correctly:  -1584.249
## % of predictable range [ (model-null)/(1-null) ]:  0.007548574
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq  pos:log_freq
##      1.25899     -0.08561     -0.07513      0.01364
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3910 Residual
## Null Deviance:      4683
## Residual Deviance: 4651  AIC: 4659
## log likelihood:  -2325.639
## Nagelkerke R2:  0.01147103
## % pres/err predicted correctly:  -1583.527
## % of predictable range [ (model-null)/(1-null) ]:  0.008000556
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq
##      1.25861     -0.08721     -0.02140

```

```

##
## Degrees of Freedom: 3913 Total (i.e. Null); 3911 Residual
## Null Deviance: 4683
## Residual Deviance: 4654 AIC: 4660
## log likelihood: -2326.916
## Nagelkerke R2: 0.01054293
## % pres/err predicted correctly: -1584.54
## % of predictable range [ (model-null)/(1-null) ]: 0.007366107
## *****
## model index: 19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 1.351626 0.007005 -0.146799
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3911 Residual
## Null Deviance: 4683
## Residual Deviance: 4654 AIC: 4660
## log likelihood: -2327.101
## Nagelkerke R2: 0.01040858
## % pres/err predicted correctly: -1584.659
## % of predictable range [ (model-null)/(1-null) ]: 0.007291756
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq pos:log_freq
## 1.10376 0.02368 -0.09275 -0.06697 0.01279
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3909 Residual
## Null Deviance: 4683
## Residual Deviance: 4650 AIC: 4660
## log likelihood: -2325.152
## Nagelkerke R2: 0.01182468
## % pres/err predicted correctly: -1583.136
## % of predictable range [ (model-null)/(1-null) ]: 0.00824547
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq
## 1.07973 0.02731 -0.09535 -0.01582
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3910 Residual
## Null Deviance: 4683

```

```

## Residual Deviance: 4653 AIC: 4661
## log likelihood: -2326.262
## Nagelkerke R2: 0.01101863
## % pres/err predicted correctly: -1584.014
## % of predictable range [ (model-null)/(1-null) ]: 0.007695562
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos
## 1.139562 0.029311 0.005467 -0.142553
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3910 Residual
## Null Deviance: 4683
## Residual Deviance: 4653 AIC: 4661
## log likelihood: -2326.319
## Nagelkerke R2: 0.01097736
## % pres/err predicted correctly: -1584.041
## % of predictable range [ (model-null)/(1-null) ]: 0.007678912
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos stimlen:pos
## 1.247056 0.007250 -0.161081 0.007727
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3910 Residual
## Null Deviance: 4683
## Residual Deviance: 4653 AIC: 4661
## log likelihood: -2326.355
## Nagelkerke R2: 0.01095096
## % pres/err predicted correctly: -1584.035
## % of predictable range [ (model-null)/(1-null) ]: 0.007682574
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq
## 1.110459 0.024551 -0.089940 -0.095533 0.009588
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3909 Residual
## Null Deviance: 4683
## Residual Deviance: 4652 AIC: 4662
## log likelihood: -2325.875
## Nagelkerke R2: 0.01129972

```

```

## % pres/err predicted correctly: -1583.696
## % of predictable range [ (model-null)/(1-null) ]: 0.007894682
## *****
## model index: 13
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos      log_freq
## 1.180298      0.024204      0.005585      -0.143482      -0.016104
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3909 Residual
## Null Deviance: 4683
## Residual Deviance: 4652 AIC: 4662
## log likelihood: -2325.97
## Nagelkerke R2: 0.01123044
## % pres/err predicted correctly: -1583.799
## % of predictable range [ (model-null)/(1-null) ]: 0.007830329
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos      log_freq I(pos^2):log_freq
## 1.378367      0.008229      -0.158025      -0.086441      -0.000470
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3908 Residual
## Null Deviance: 4683
## Residual Deviance: 4650 AIC: 4662
## log likelihood: -2324.982
## Nagelkerke R2: 0.01194877
## % pres/err predicted correctly: -1583.054
## % of predictable range [ (model-null)/(1-null) ]: 0.008296402
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq          pos      stimlen:log_freq      log_fr
## 1.112604      0.022980      -0.088281      -0.092995      0.003333      0.0
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3908 Residual
## Null Deviance: 4683
## Residual Deviance: 4650 AIC: 4662
## log likelihood: -2325.114
## Nagelkerke R2: 0.01185252
## % pres/err predicted correctly: -1583.101
## % of predictable range [ (model-null)/(1-null) ]: 0.008266835
## *****

```



```

## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          log_freq          I(pos^2)          pos          stimlen:log
## 1.219143          0.021113          -0.094047          0.005947          -0.146809          0.
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3908 Residual
## Null Deviance: 4683
## Residual Deviance: 4651 AIC: 4663
## log likelihood: -2325.545
## Nagelkerke R2: 0.01153925
## % pres/err predicted correctly: -1583.458
## % of predictable range [ (model-null)/(1-null) ]: 0.008043511
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          I(pos^2)          pos          log_freq          I(pos
## 1.2354329          0.0194617          0.0071527          -0.1544930          -0.0803651
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3907 Residual
## Null Deviance: 4683
## Residual Deviance: 4649 AIC: 4663
## log likelihood: -2324.664
## Nagelkerke R2: 0.01217954
## % pres/err predicted correctly: -1582.792
## % of predictable range [ (model-null)/(1-null) ]: 0.008460243
## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          I(pos^2)          pos          stimlen:I(pos^2)          stimle
## 0.980732          0.044787          -0.019952          0.007793          0.002651          -0.
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3908 Residual
## Null Deviance: 4683
## Residual Deviance: 4652 AIC: 4664
## log likelihood: -2326.145
## Nagelkerke R2: 0.01110358
## % pres/err predicted correctly: -1583.908
## % of predictable range [ (model-null)/(1-null) ]: 0.007762152
## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
##      (Intercept)          stimlen          log_freq          I(pos^2)          pos          stim
##      1.2451087          0.0187099          -0.1068483          0.0071249          -0.1546390
##      log_freq:pos
##      0.0193325
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3906 Residual
## Null Deviance: 4683
## Residual Deviance: 4649 AIC: 4665
## log likelihood: -2324.617
## Nagelkerke R2: 0.01221322
## % pres/err predicted correctly: -1582.756
## % of predictable range [ (model-null)/(1-null) ]: 0.008483349
## *****
## model index: 14
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      0.9166
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3913 Residual
## Null Deviance: 4683
## Residual Deviance: 4683 AIC: 4685
## log likelihood: -2341.365
## Nagelkerke R2: -3.182426e-16
## % pres/err predicted correctly: -1596.306
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 15
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen
##      1.04811          -0.01702
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3912 Residual
## Null Deviance: 4683
## Residual Deviance: 4682 AIC: 4686
## log likelihood: -2341.034
## Nagelkerke R2: 0.000242303
## % pres/err predicted correctly: -1596.036
## % of predictable range [ (model-null)/(1-null) ]: 0.0001692916
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)

```

```
##
## Coefficients:
## (Intercept)      stimlen      log_freq
##      1.08748      -0.02211      -0.01638
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3911 Residual
## Null Deviance:      4683
## Residual Deviance: 4681 AIC: 4687
## log likelihood: -2340.671
## Nagelkerke R2:  0.0005082854
## % pres/err predicted correctly: -1595.739
## % of predictable range [ (model-null)/(1-null) ]:  0.0003549142
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq stimlen:log_freq
##      1.115913      -0.024748      -0.084995      0.008868
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3910 Residual
## Null Deviance:      4683
## Residual Deviance: 4681 AIC: 4689
## log likelihood: -2340.338
## Nagelkerke R2:  0.0007520913
## % pres/err predicted correctly: -1595.473
## % of predictable range [ (model-null)/(1-null) ]:  0.0005214723
## *****
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                      by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                              RunLabel,"_",CurTask,
                              "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	log_stimlen	log_pos	log_freq	log_freq^2	log_freq:log_stimlen	log_freq:log_pos	log_freq:log_pos^2	len:I(pos^2)
preserved ~ pos	4659.148	0.0000000	0.0000000	0.3202160	0.0002597836	NA	NA	-	NA	NA	NA	NA	NA	NA
											0.0852246			

Model	AIC Delta	AIC	AICw	NagR	Intercept	log_freq	stimlen	log_pos	log_freq	I(pos)	pos^2	log_freq	I(pos)	pos^2
preserved ~ stimlen + pos	4659.09	10801975	0.872885	0.407041	1782322601	NA	-	NA	NA	NA	NA	NA	NA	NA
preserved ~ pos *	4659.27	10902256	0.352865	0.412589	17838863	-	NA	-	0.013642	NA	NA	NA	NA	NA
log_freq					0.0751302		0.0856091							
preserved ~ pos + log_freq	4659.83	11506739	0.073731	0.542380	17860069	-	NA	-	NA	NA	NA	NA	NA	NA
preserved ~ pos + log_freq					0.0214006		0.0872138							
preserved ~ I(pos^2) + pos	4660.20	12465790	0.797918	0.210861	178616260	NA	NA	-	NA	NA	0.0070035	NA	NA	NA
preserved ~ stimlen + pos *	4660.80	15723506	0.720228	0.824037	17862236785	NA	-	0.0127947	NA	NA	NA	NA	NA	NA
log_freq					0.0669686		0.0927481							
preserved ~ stimlen + pos + log_freq	4660.52	16033025	0.466348	0.108703	1786233053	NA	-	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + pos + log_freq					0.0158202		0.0953534							
preserved ~ stimlen + I(pos^2) + pos	4660.63	18971454	0.268509	0.739562	17862293140	NA	-	NA	NA	0.0054673	NA	NA	NA	NA
preserved ~ stimlen * pos	4660.75	16286357	0.604004	0.095247	17860572149	NA	-	NA	NA	NA	NA	0.0077273	NA	NA
preserved ~ stimlen *	4661.75	16049272	0.035936	0.129970	1786245508	0.0095877	NA	NA	NA	NA	NA	NA	NA	NA
log_freq + pos					0.0899402		0.0955331							
preserved ~ stimlen + I(pos^2) + pos + log_freq	4661.24	19309237	0.378267	0.123080	1786297242041	NA	-	NA	NA	0.0055348	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos) *	4661.26	15632146	0.392305	0.194883	17863673	-	NA	-	0.0190115	0.0082293	NA	NA	NA	NA
log_freq					0.0864414		0.1580253			0.0004700				
preserved ~ stimlen *	4662.22	18067174	0.128295	0.152260	1786229801	0.0033329	NA	0.0116814	NA	NA	NA	NA	NA	NA
log_freq + pos *					0.0882805		0.0929947							
log_freq														
preserved ~ stimlen *	4663.09	14301029	0.012381	0.152190	1786211134	0.0100773	NA	NA	0.0059472	NA	NA	NA	NA	NA
log_freq + I(pos^2) + pos					0.0940473		0.1468086							

Model	AIC Delta	AIC	AICw	NagR ²	Intercept	log_freq	stimlen	log_pos	log_freq(I(pos^2))	pos^2	log_freq(I(pos^2))	pos^2	log_freq(I(pos^2))	pos^2
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	4663.428	9965236	8933302	1.235	4329	4617	NA	-	0.0189	NA	0.0071527	NA	NA	NA
					0.0803651	0.1544930			0.0005554					
preserved ~ stimlen * (I(pos^2) + pos)	4664.294	2841764	4600928	1.098	7304	4786	NA	0.0077930	NA	-	NA	NA	-	0.0026508
										0.0199517			0.0153694	
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	4665.635	7209876	6282922	1.235	4329	4617	0.0037173	NA	0.0193307	NA	-	NA	NA	
							0.1068483	0.1546390			0.0007456			
preserved ~ 1	4684.235	5829089	002800000	0.000	6484	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen	4686.269	2021200	0000002	1.428	1056	NA	NA	NA	NA	NA	NA	NA	NA	NA
					0.0170186									
preserved ~ stimlen + log_freq	4687.281	2940337	000800005	0.087	4773	-	NA	NA	NA	NA	NA	NA	NA	NA
					0.0220150	0.63815								
preserved ~ stimlen * log_freq	4688.297	6283380	000000075	1.591	28	-	0.0088081	NA	NA	NA	NA	NA	NA	NA
					0.0240484	0.9949								

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ pos"
```

```
print(BestFLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      1.25078      -0.08522
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3912 Residual
## Null Deviance:      4683
## Residual Deviance: 4655  AIC: 4659
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```
PosDat$FLPFitted <- fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

```

HF_Plot <- plot_len_pos_obs_predicted(HFdat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

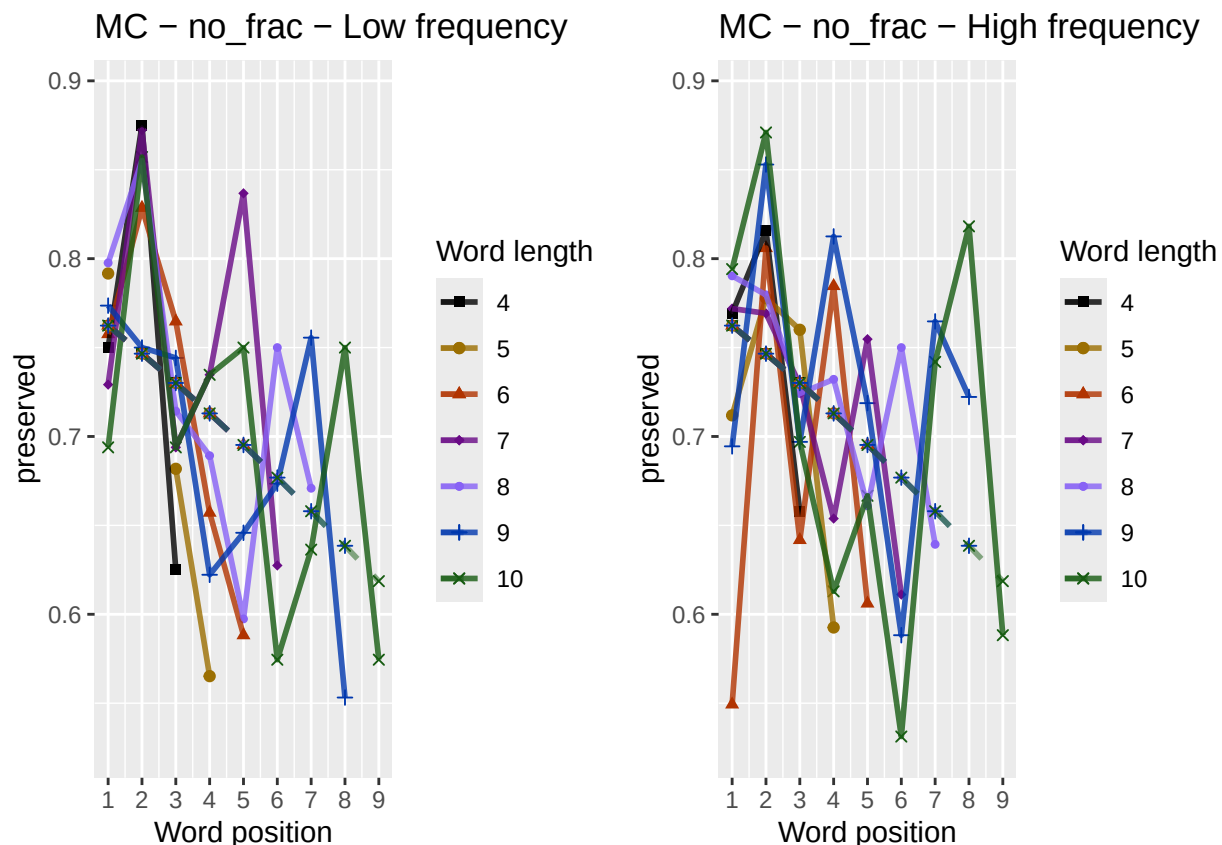
LF_Plot <- plot_len_pos_obs_predicted(LFdat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)

## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)

```



```

# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",

```

```

"preserved ~ pos",
"preserved ~ stimlen",
"preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 1.2409 -0.4417
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3912 Residual
## Null Deviance: 4683
## Residual Deviance: 4527 AIC: 4531
## log likelihood: -2263.273
## Nagelkerke R2: 0.05606594
## % pres/err predicted correctly: -1529.683
## % of predictable range [ (model-null)/(1-null) ]: 0.0417094
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 1.25078 -0.08522
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3912 Residual
## Null Deviance: 4683
## Residual Deviance: 4655 AIC: 4659
## log likelihood: -2327.574
## Nagelkerke R2: 0.01006489
## % pres/err predicted correctly: -1585.017
## % of predictable range [ (model-null)/(1-null) ]: 0.007067415
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 1.351626 0.007005 -0.146799
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3911 Residual
## Null Deviance: 4683
## Residual Deviance: 4654 AIC: 4660
## log likelihood: -2327.101

```

```

## Nagelkerke R2: 0.01040858
## % pres/err predicted correctly: -1584.659
## % of predictable range [ (model-null)/(1-null) ]: 0.007291756
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 0.78606 0.06561
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3912 Residual
## Null Deviance: 4683
## Residual Deviance: 4672 AIC: 4676
## log likelihood: -2336.06
## Nagelkerke R2: 0.003880328
## % pres/err predicted correctly: -1592.115
## % of predictable range [ (model-null)/(1-null) ]: 0.002624074
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 0.9166
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3913 Residual
## Null Deviance: 4683
## Residual Deviance: 4683 AIC: 4685
## log likelihood: -2341.365
## Nagelkerke R2: -3.182426e-16
## % pres/err predicted correctly: -1596.306
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 1.04811 -0.01702
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3912 Residual
## Null Deviance: 4683
## Residual Deviance: 4682 AIC: 4686
## log likelihood: -2341.034
## Nagelkerke R2: 0.000242303
## % pres/err predicted correctly: -1596.036
## % of predictable range [ (model-null)/(1-null) ]: 0.0001692916

```



```
## *****
```

```
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	4530.5460	0.0000	1	1	0.0560659	0.2409454	NA	-	NA	NA	NA
preserved ~ pos	4659.1481	28.6016	0	0	0.0100649	0.2507836	NA	NA	NA	-	NA
										0.0852246	
preserved ~ (I(pos^2) + pos)	4660.2021	29.6562	0	0	0.0104086	0.3516260	NA	NA	0.0070055	-	NA
										0.1467994	
preserved ~ CumPres	4676.1201	45.5735	0	0	0.0038800	0.7860557	0.065608	NA	NA	NA	NA
preserved ~ 1	4684.7311	54.1845	0	0	0.0000000	0.9166484	NA	NA	NA	NA	NA
preserved ~ stimlen	4686.0691	55.5228	0	0	0.0002423	0.0481056	NA	NA	NA	NA	-
											0.0170186

```
if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
}
```

```

AICValues <- c(BestMEModel$aic,RndModelAIC)
BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

write.csv(BestMEModelRndDF,
           paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                  "_best_main_effects_model_with_random_cum_term.csv"),
           row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                         RunLabel,"_",CurTask,
                                         "_syllable_component_summary.csv"),row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.6561265	506
O	0.6065574	1830
P	0.5625000	32
S	0.6623377	231
V	0.8996198	1315

```

# main effects models for data without satellite positions

keep_components = c("0","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.2567      -0.4867
##

```

```

## Degrees of Freedom: 3650 Total (i.e. Null); 3649 Residual
## Null Deviance: 4337
## Residual Deviance: 4183 AIC: 4187
## log likelihood: -2091.623
## Nagelkerke R2: 0.0592328
## % pres/err predicted correctly: -1409.076
## % of predictable range [ (model-null)/(1-null) ]: 0.04424255
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 1.30729 -0.09301
##
## Degrees of Freedom: 3650 Total (i.e. Null); 3649 Residual
## Null Deviance: 4337
## Residual Deviance: 4306 AIC: 4310
## log likelihood: -2153.144
## Nagelkerke R2: 0.01195395
## % pres/err predicted correctly: -1462.02
## % of predictable range [ (model-null)/(1-null) ]: 0.008356815
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 1.42898 0.00839 -0.16694
##
## Degrees of Freedom: 3650 Total (i.e. Null); 3648 Residual
## Null Deviance: 4337
## Residual Deviance: 4305 AIC: 4311
## log likelihood: -2152.514
## Nagelkerke R2: 0.0124469
## % pres/err predicted correctly: -1461.56
## % of predictable range [ (model-null)/(1-null) ]: 0.008668847
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 0.8325 0.0562
##
## Degrees of Freedom: 3650 Total (i.e. Null); 3649 Residual
## Null Deviance: 4337
## Residual Deviance: 4330 AIC: 4334

```

```
## log likelihood: -2164.971
## Nagelkerke R2: 0.002681335
## % pres/err predicted correctly: -1471.677
## % of predictable range [ (model-null)/(1-null) ]: 0.001811344
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 0.9394
##
## Degrees of Freedom: 3650 Total (i.e. Null); 3650 Residual
## Null Deviance: 4337
## Residual Deviance: 4337 AIC: 4339
## log likelihood: -2168.377
## Nagelkerke R2: 0
## % pres/err predicted correctly: -1474.349
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 0.987768 -0.006263
##
## Degrees of Freedom: 3650 Total (i.e. Null); 3649 Residual
## Null Deviance: 4337
## Residual Deviance: 4337 AIC: 4341
## log likelihood: -2168.335
## Nagelkerke R2: 3.279162e-05
## % pres/err predicted correctly: -1474.315
## % of predictable range [ (model-null)/(1-null) ]: 2.286569e-05
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	4187.240	0.0000	1	1	0.0592328	8.2567368	NA	-	NA	NA	NA
preserved ~ pos	4310.289	123.0433	0	0	0.0119539	9.3072919	NA	NA	NA	-	NA
										0.0930141	
preserved ~ (I(pos^2) + pos)	4311.027	123.7815	0	0	0.0124469	9.4289754	NA	NA	0.00839	-	NA
										0.1669397	

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumPres	4333.942	46.6964	0	0	0.002681	0.8324577	0.0561959	NA	NA	NA	NA
preserved ~ 1	4338.753	51.5076	0	0	0.0000000	0.9394131	NA	NA	NA	NA	NA
preserved ~ stimlen	4340.670	53.4244	0	0	0.0000328	0.9877684	NA	NA	NA	NA	- 0.0062628

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.2214      -0.4434
##
## Degrees of Freedom: 3144 Total (i.e. Null); 3143 Residual
## Null Deviance:      3674
## Residual Deviance: 3598 AIC: 3602
## log likelihood: -1799.149
## Nagelkerke R2: 0.03467238
## % pres/err predicted correctly: -1209.43
## % of predictable range [ (model-null)/(1-null) ]: 0.02571363
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      1.37549      -0.09765
##
## Degrees of Freedom: 3144 Total (i.e. Null); 3143 Residual
## Null Deviance:      3674
## Residual Deviance: 3644 AIC: 3648
## log likelihood: -1822.181
## Nagelkerke R2: 0.01377332
```

```

## % pres/err predicted correctly: -1229.422
## % of predictable range [ (model-null)/(1-null) ]: 0.009622067
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 1.407817 0.002301 -0.117844
##
## Degrees of Freedom: 3144 Total (i.e. Null); 3142 Residual
## Null Deviance: 3674
## Residual Deviance: 3644 AIC: 3650
## log likelihood: -1822.138
## Nagelkerke R2: 0.01381286
## % pres/err predicted correctly: -1229.401
## % of predictable range [ (model-null)/(1-null) ]: 0.009638505
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 0.99
##
## Degrees of Freedom: 3144 Total (i.e. Null); 3144 Residual
## Null Deviance: 3674
## Residual Deviance: 3674 AIC: 3676
## log likelihood: -1837.177
## Nagelkerke R2: -3.222194e-16
## % pres/err predicted correctly: -1241.376
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 0.95992 0.01845
##
## Degrees of Freedom: 3144 Total (i.e. Null); 3143 Residual
## Null Deviance: 3674
## Residual Deviance: 3674 AIC: 3678
## log likelihood: -1836.938
## Nagelkerke R2: 0.0002205344
## % pres/err predicted correctly: -1241.189
## % of predictable range [ (model-null)/(1-null) ]: 0.0001504153
## *****

```

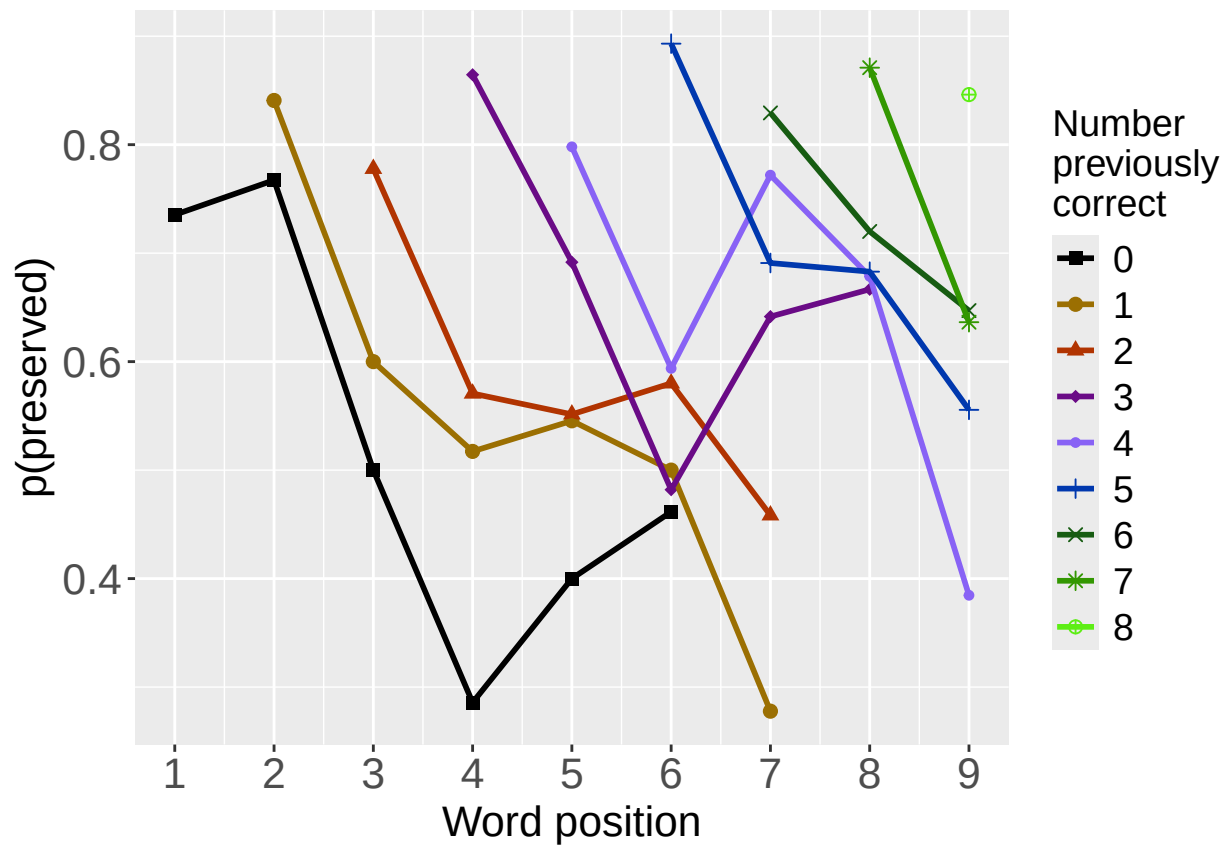
```
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##  0.9960637   -0.0007843
##
## Degrees of Freedom: 3144 Total (i.e. Null);  3143 Residual
## Null Deviance:      3674
## Residual Deviance: 3674 AIC: 3678
## log likelihood:  -1837.177
## Nagelkerke R2:  5.154617e-07
## % pres/err predicted correctly:  -1241.375
## % of predictable range [ (model-null)/(1-null) ]:  3.574265e-07
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	3602.298	0.00000	1	1	0.034672	1.2213895	NA	-	NA	NA	NA
preserved ~ pos	3648.362	46.06322	0	0	0.013773	3.3754931	NA	NA	NA	-	NA
										0.0976499	
preserved ~ (I(pos^2) + pos)	3650.275	47.97671	0	0	0.013812	3.4078174	NA	NA	0.0023005	-	NA
										0.1178443	
preserved ~ 1	3676.354	74.05603	0	0	0.0000000	0.9900298	NA	NA	NA	NA	NA
preserved ~ CumPres	3677.876	75.57804	0	0	0.0002206	0.9599204	0.0184457	NA	NA	NA	NA
preserved ~ stimlen	3678.353	76.05491	0	0	0.0000000	0.9960637	NA	NA	NA	NA	-
											0.0007843

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

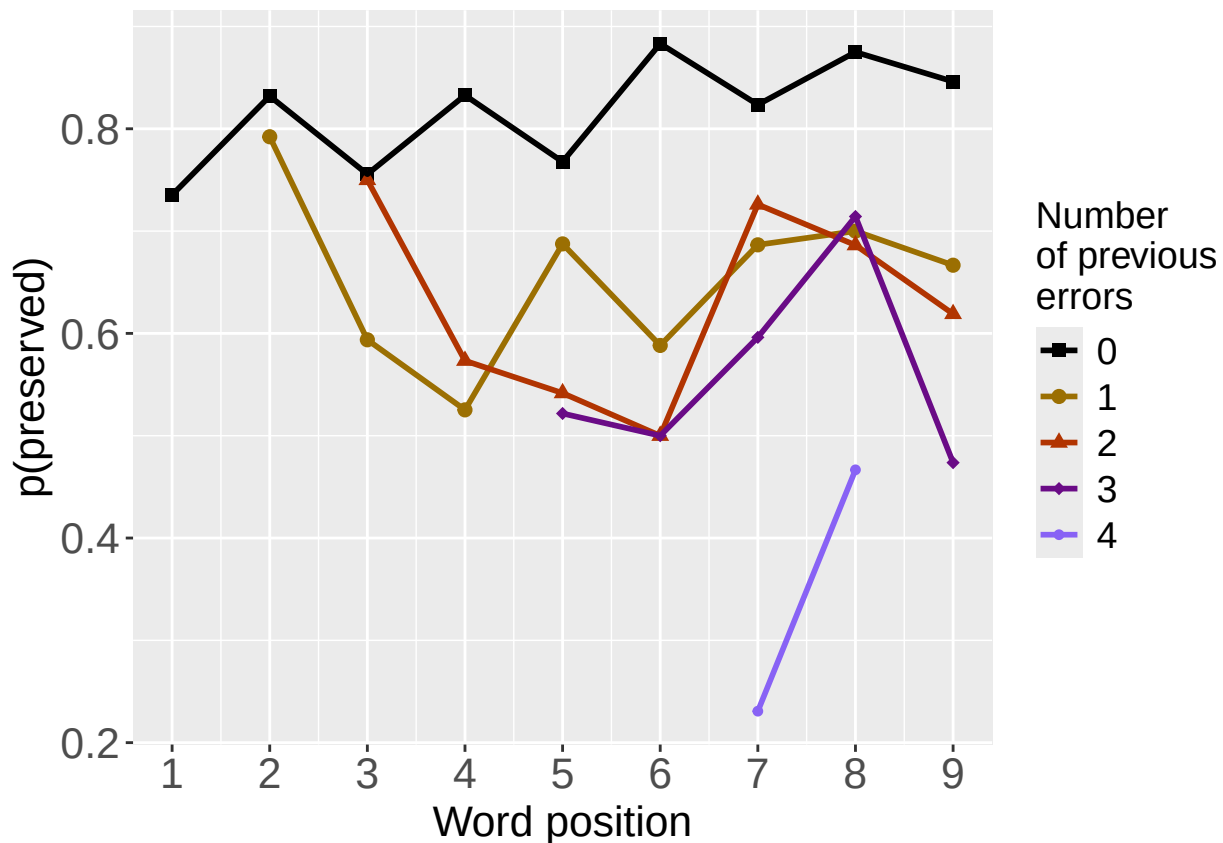
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

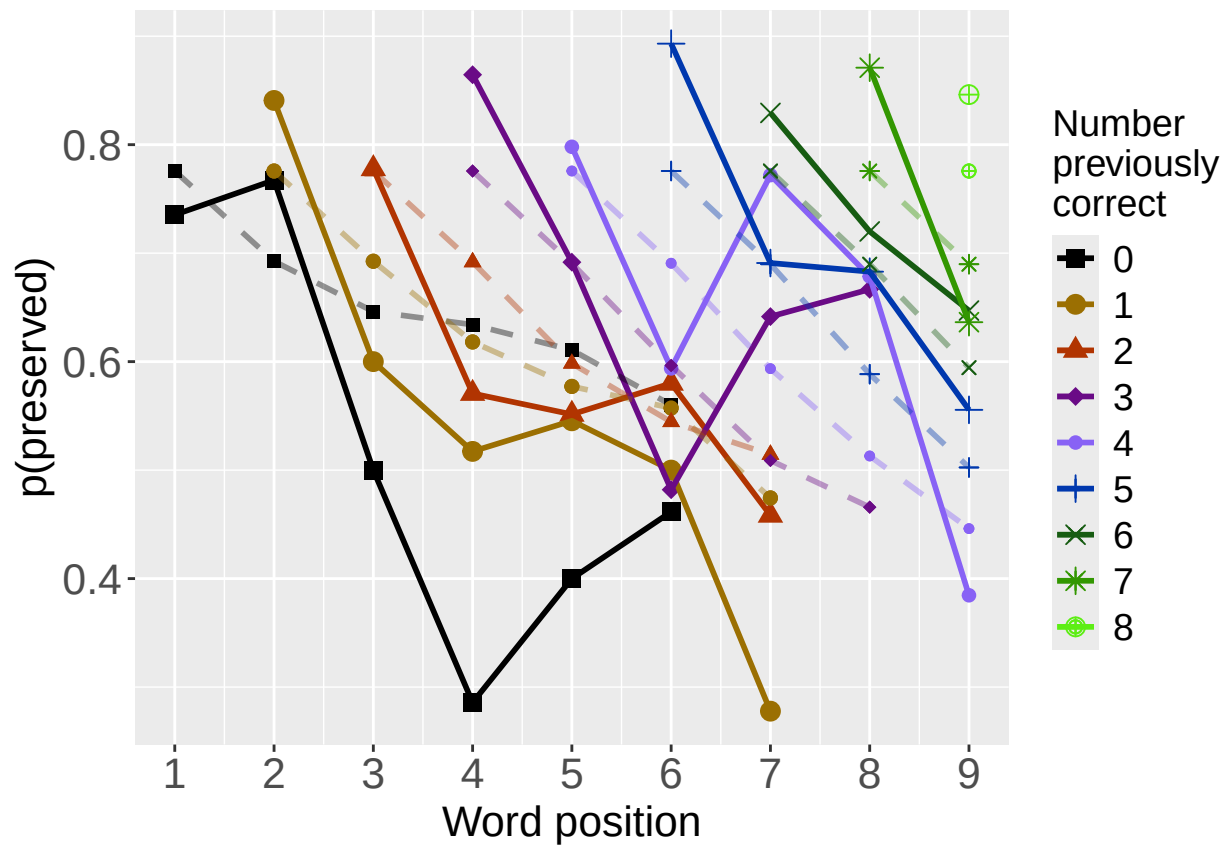
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

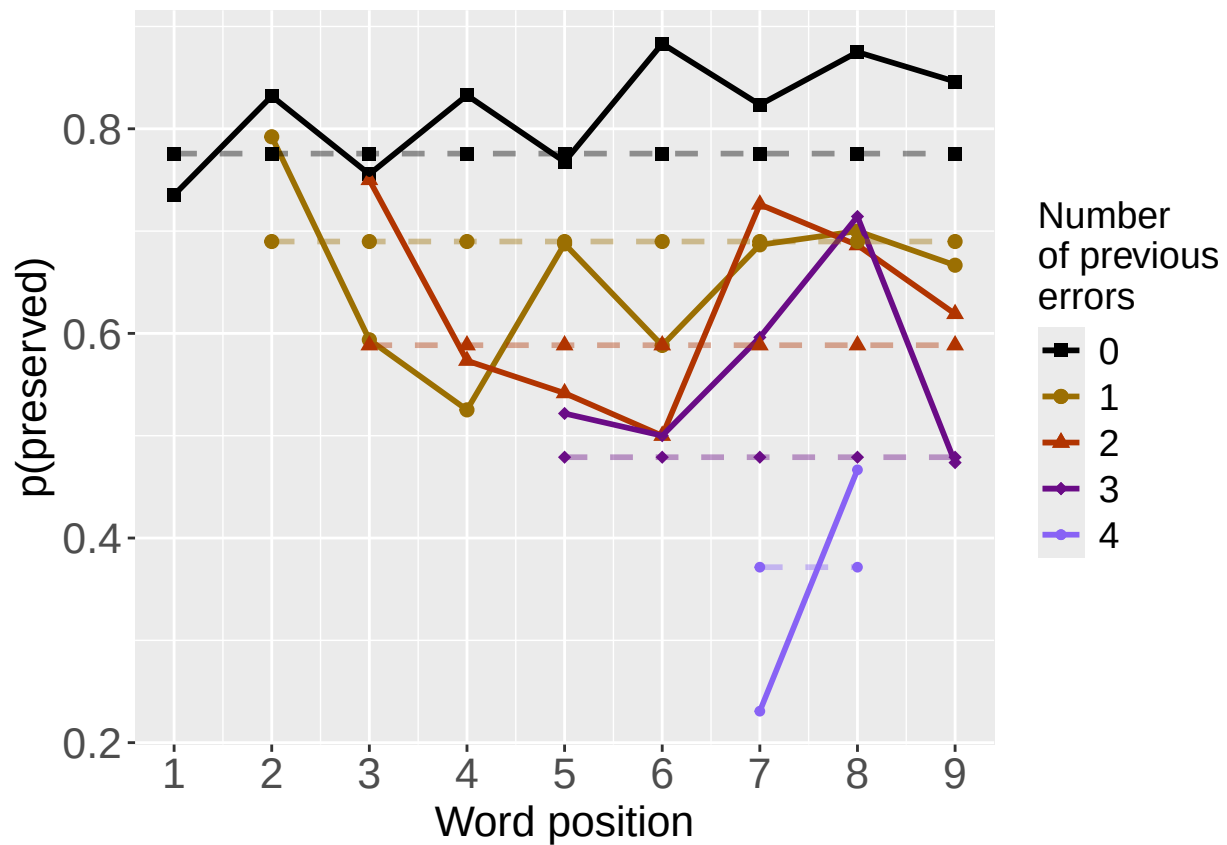
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)          pos
##    1.26624    -0.51487     0.01294    -0.05866
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3910 Residual
## Null Deviance:      4683
## Residual Deviance: 4517  AIC: 4525
## log likelihood:  -2258.521
## Nagelkerke R2:  0.0594063
## % pres/err predicted correctly:  -1525.951
## % of predictable range [ (model-null)/(1-null) ]:  0.04404585
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.2409      -0.4417
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3912 Residual
## Null Deviance:      4683
## Residual Deviance: 4527  AIC: 4531
## log likelihood:  -2263.273
## Nagelkerke R2:  0.05606594
## % pres/err predicted correctly:  -1529.683
## % of predictable range [ (model-null)/(1-null) ]:  0.0417094
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      1.351626      0.007005      -0.146799
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3911 Residual
## Null Deviance:      4683
## Residual Deviance: 4654  AIC: 4660
## log likelihood:  -2327.101
## Nagelkerke R2:  0.01040858
## % pres/err predicted correctly:  -1584.659
## % of predictable range [ (model-null)/(1-null) ]:  0.007291756
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	4525.041	0.00000	1.000000	0.9400555	0.0594063	1.266240	-0.5148732	0.0129434	-0.0586603
preserved ~ CumErr	4530.546	5.50504	0.063767	0.0599445	0.0560659	1.240945	-0.4416668	NA	NA
preserved ~ I(pos^2) + pos	4660.202	135.16127	0.000000	0.0000000	0.0104086	1.351626	NA	0.0070055	-0.1467994

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 0.8765 -0.4621 0.0492
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3911 Residual
## Null Deviance: 4683
## Residual Deviance: 4522 AIC: 4528
## log likelihood: -2260.782
## Nagelkerke R2: 0.05781808
## % pres/err predicted correctly: -1527.471
## % of predictable range [ (model-null)/(1-null) ]: 0.04309422
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 1.2409 -0.4417
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3912 Residual
## Null Deviance: 4683
## Residual Deviance: 4527 AIC: 4531
## log likelihood: -2263.273
## Nagelkerke R2: 0.05606594
## % pres/err predicted correctly: -1529.683
## % of predictable range [ (model-null)/(1-null) ]: 0.0417094
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      1.04811      -0.01702
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3912 Residual
## Null Deviance:      4683
## Residual Deviance: 4682 AIC: 4686
## log likelihood:  -2341.034
## Nagelkerke R2:   0.000242303
## % pres/err predicted correctly:  -1596.036
## % of predictable range [ (model-null)/(1-null) ]:  0.0001692916
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr + stimlen	4527.563	0.000000	1.0000000	0.8162933	0.0578181	0.8765322	- 0.4621198	0.0492036
preserved ~ CumErr	4530.546	2.982867	0.2250498	0.1837067	0.0560659	1.2409454	- 0.4416668	NA
preserved ~ stimlen	4686.069	158.505648	0.0000000	0.0000000	0.0002423	1.0481056	NA	- 0.0170186

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      1.06156      -0.46234      0.09804
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3911 Residual
## Null Deviance:      4683
## Residual Deviance: 4505 AIC: 4511
## log likelihood:  -2252.498
## Nagelkerke R2:   0.06362734
## % pres/err predicted correctly:  -1521.635
## % of predictable range [ (model-null)/(1-null) ]:  0.04674824
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 1.2409 -0.4417
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3912 Residual
## Null Deviance: 4683
## Residual Deviance: 4527 AIC: 4531
## log likelihood: -2263.273
## Nagelkerke R2: 0.05606594
## % pres/err predicted correctly: -1529.683
## % of predictable range [ (model-null)/(1-null) ]: 0.0417094
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 0.78606 0.06561
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3912 Residual
## Null Deviance: 4683
## Residual Deviance: 4672 AIC: 4676
## log likelihood: -2336.06
## Nagelkerke R2: 0.003880328
## % pres/err predicted correctly: -1592.115
## % of predictable range [ (model-null)/(1-null) ]: 0.002624074
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	4510.997	0.00000	1.00e+00	0.9999431	0.0636273	1.0615589	- 0.4623434	0.098039
preserved ~ CumErr	4530.546	19.54916	5.69e-05	0.0000569	0.0560659	1.2409454	- 0.4416668	NA
preserved ~ CumPres	4676.120	165.12262	0.00e+00	0.0000000	0.0038803	0.7860557	NA	0.065608

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      1.08533      -0.51030      0.05284
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3911 Residual
## Null Deviance:      4683
## Residual Deviance: 4520  AIC: 4526
## log likelihood:  -2260.039
## Nagelkerke R2:  0.05833963
## % pres/err predicted correctly:  -1527.397
## % of predictable range [ (model-null)/(1-null) ]:  0.04314105
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.2409      -0.4417
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3912 Residual
## Null Deviance:      4683
## Residual Deviance: 4527  AIC: 4531
## log likelihood:  -2263.273
## Nagelkerke R2:  0.05606594
## % pres/err predicted correctly:  -1529.683
## % of predictable range [ (model-null)/(1-null) ]:  0.0417094
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      1.25078      -0.08522
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3912 Residual
## Null Deviance:      4683
## Residual Deviance: 4655  AIC: 4659
## log likelihood:  -2327.574
## Nagelkerke R2:  0.01006489
## % pres/err predicted correctly:  -1585.017
## % of predictable range [ (model-null)/(1-null) ]:  0.007067415
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr + pos	4526.079	0.000000	1.000000	0.9032318	0.0583396	1.085332	- 0.5103023	0.0528402
preserved ~ CumErr	4530.546	4.467322	0.1071355	0.0967682	0.0560659	1.240945	- 0.4416668	NA
preserved ~ pos	4659.148	133.068891	0.000000	0.000000	0.0100649	1.250784	NA	- 0.0852246

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + CumPres	4510.997	0.000000	1.000000	0.9999431	0.0636273	0.0615589	- 0.4623434	NA	NA	NA	0.098039
preserved ~ CumErr + I(pos^2) + pos	4525.040	0.000000	1.000000	0.9400535	0.0594063	0.2662398	- 0.5148732	0.0129434	-	NA	NA
preserved ~ CumErr + pos	4526.079	0.000000	1.000000	0.9032318	0.0583396	0.0853321	- 0.5103023	NA	0.0528402	NA	NA
preserved ~ CumErr + stimlen	4527.568	0.000000	1.000000	0.8162933	0.0578181	0.18765322	- 0.4621198	NA	NA	0.0492036	NA
preserved ~ CumErr	4530.546	5.050400	0.0637670	0.0599445	0.0560659	0.2409454	- 0.4416668	NA	NA	NA	NA
preserved ~ CumErr	4530.548	9.982867	0.2250498	0.1837067	0.0560659	0.2409454	- 0.4416668	NA	NA	NA	NA
preserved ~ CumErr	4530.546	9.549164	0.0000569	0.0000056	0.0560659	0.2409454	- 0.4416668	NA	NA	NA	NA
preserved ~ CumErr	4530.548	4.673220	0.1071355	0.0967682	0.0560659	0.2409454	- 0.4416668	NA	NA	NA	NA
preserved ~ pos	4659.148	133.068891	0.000000	0.000000	0.0100649	0.2507836	NA	NA	- 0.0852246	NA	NA
preserved ~ I(pos^2) + pos	4660.202	35.161266	0.000000	0.000000	0.0104086	0.3516260	NA	0.0070055	- 0.1467994	NA	NA
preserved ~ CumPres	4676.120	65.122608	0.000000	0.000000	0.0038803	0.7860557	NA	NA	NA	NA	0.065608
preserved ~ stimlen	4686.069	58.505643	0.000000	0.000000	0.0002423	0.481056	NA	NA	NA	- 0.0170186	NA

```
# explore influence of frequency and length

if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      1.06156      -0.46234      0.09804
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3911 Residual
## Null Deviance:      4683
## Residual Deviance: 4505 AIC: 4511
## log likelihood: -2252.498
## Nagelkerke R2: 0.06362734
## % pres/err predicted correctly: -1521.635
## % of predictable range [ (model-null)/(1-null) ]: 0.04674824
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen
##      0.93980      -0.46848      0.09264      0.01775
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3910 Residual
## Null Deviance:      4683
## Residual Deviance: 4504 AIC: 4512
## log likelihood: -2252.206
## Nagelkerke R2: 0.0638323
## % pres/err predicted correctly: -1521.27
## % of predictable range [ (model-null)/(1-null) ]: 0.04697659
## *****
## model index: 3

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      1.06524      -0.46319      0.09650      -0.01314
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3910 Residual
## Null Deviance:      4683
## Residual Deviance: 4505 AIC: 4513
## log likelihood: -2252.259
## Nagelkerke R2:  0.06379466
## % pres/err predicted correctly: -1521.341
## % of predictable range [ (model-null)/(1-null) ]:  0.04693249
## *****
## model index:  5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen      log_freq
##      0.96414      -0.46805      0.09240      0.01462      -0.01010
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3909 Residual
## Null Deviance:      4683
## Residual Deviance: 4504 AIC: 4514
## log likelihood: -2252.074
## Nagelkerke R2:  0.06392465
## % pres/err predicted correctly: -1521.109
## % of predictable range [ (model-null)/(1-null) ]:  0.04707743
## *****
## model index:  2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      0.9166
##
## Degrees of Freedom: 3913 Total (i.e. Null);  3913 Residual
## Null Deviance:      4683
## Residual Deviance: 4683 AIC: 4685
## log likelihood: -2341.365
## Nagelkerke R2: -3.182426e-16
## % pres/err predicted correctly: -1596.306
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

```

```

BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

```

```

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres	log_freq	stimlen
preserved ~ CumErr + CumPres	4510.997	0.0000001	0.0000000	0.4614077	0.0636273	0.615589	-	0.0980390	NA	NA
preserved ~ CumErr + CumPres + stimlen	4512.411	1.4142410	0.4930619	0.2275026	0.0638323	0.9397963	-	0.0926422	NA	0.0177549
preserved ~ CumErr + CumPres + log_freq	4512.519	1.5218210	0.4672408	0.2155885	0.0637947	0.652375	-	0.0964962	-	NA
preserved ~ CumErr + CumPres + stimlen + log_freq	4514.143	3.1502870	0.2069779	0.0955012	0.0639246	0.9641386	-	0.0924049	-	0.0146194
preserved ~ 1	4684.731	173.7337010	0.0000000	0.0000000	0.0000000	0.9166484	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

```

```

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumErr + CumPres
##      Df Deviance    AIC
## CumErr  1   4672.1 4676.1
## CumPres  1   4526.5 4530.5
## <none>    1   4505.0 4511.0

```

```

#####
# Single deletions from best model
#####

```

```

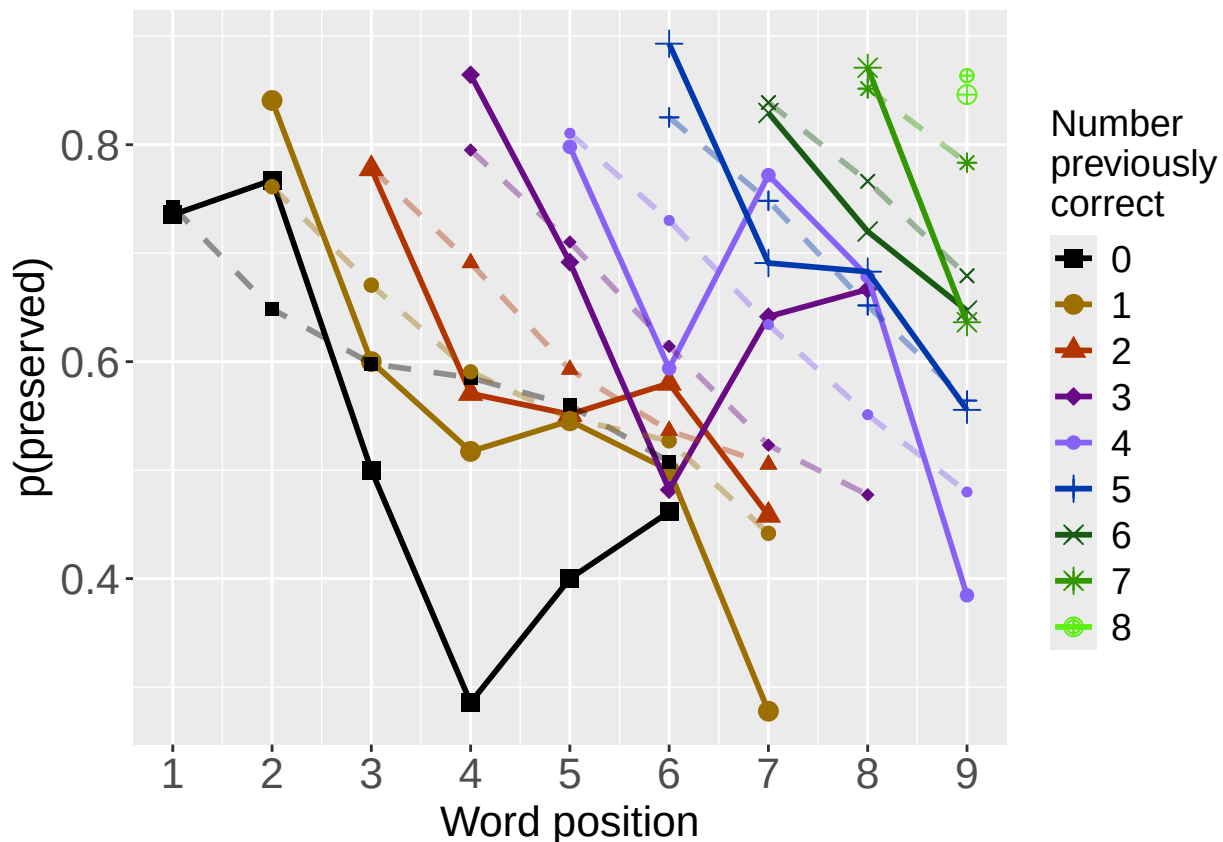
write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
         row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```

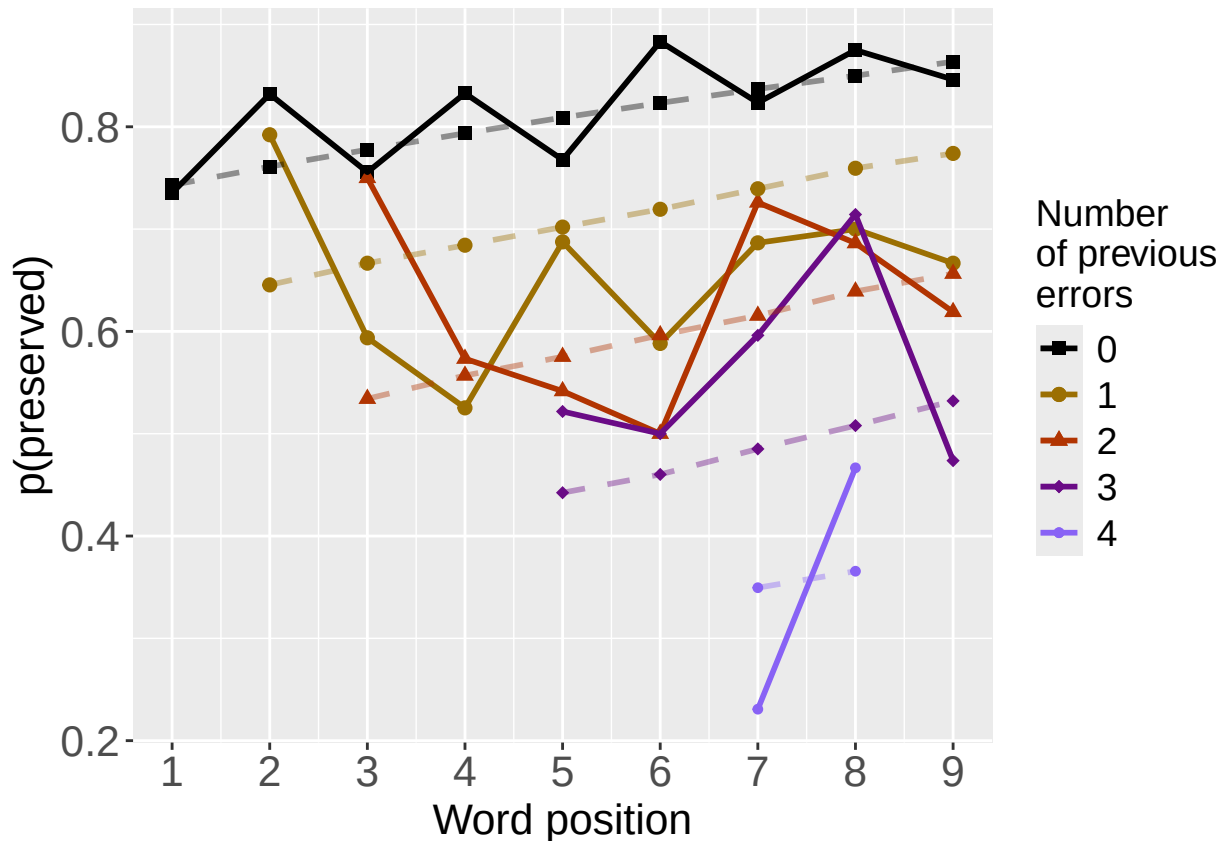


```

PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)

```



```

PlotName<-paste0(FigDir,CurPat,"_",
  RunLabel,"_",CurTask,
  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      1.2409      -0.4417
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3912 Residual
## Null Deviance: 4683
## Residual Deviance: 4527 AIC: 4531
## log likelihood: -2263.273
## Nagelkerke R2: 0.05606594
## % pres/err predicted correctly: -1529.683
## % of predictable range [ (model-null)/(1-null) ]: 0.0417094

```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      1.06156      -0.46234      0.09804
##
## Degrees of Freedom: 3913 Total (i.e. Null); 3911 Residual
## Null Deviance:      4683
## Residual Deviance: 4505 AIC: 4511
## log likelihood: -2252.498
## Nagelkerke R2: 0.06362734
## % pres/err predicted correctly: -1521.635
## % of predictable range [ (model-null)/(1-null) ]: 0.04674824
## *****

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

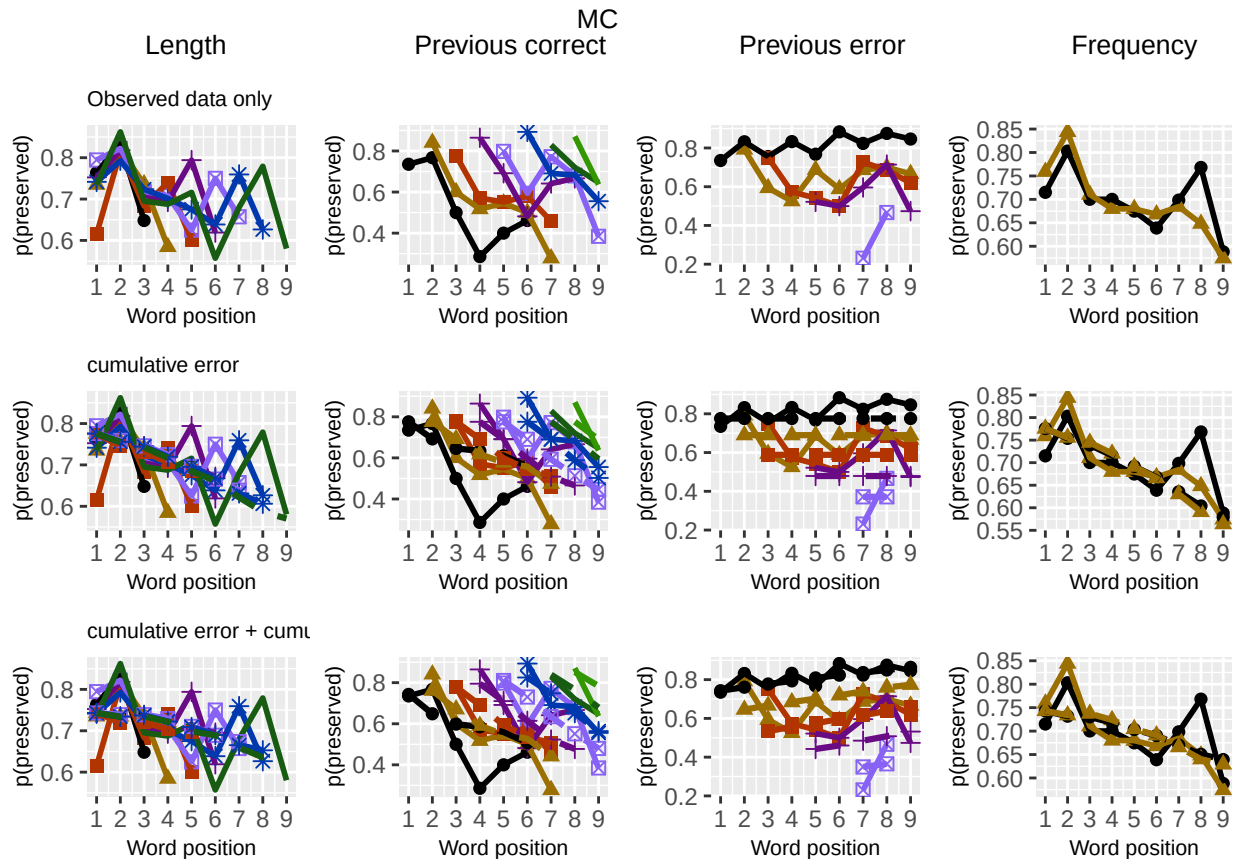
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

```

```
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)

ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
CurTask,"_dominance_analysis_table.csv"),
row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	CumPres
McFadden	0.0345212	0.0034339
SquaredCorrelation	0.0404026	0.0039916
Nagelkerke	0.0404026	0.0039916
Estrella	0.0411502	0.0040885

	deviance	deviance_explained
CumErr + CumPres	4504.997	177.7337
CumErr	4526.546	156.1845
null	4682.731	0.0000

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + CumPres	3.795514	100.00000	12.12441
CumErr	3.335330	87.87559	87.87559
null	0.000000	NA	0.00000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##          model deviance deviance_explained percent_explained percent_of_explained_deviance increment_in_explained
## CumErr + CumPres CumErr + CumPres 4504.997          177.7337          3.795514          100.00000          12.12441
## CumErr          CumErr 4526.546          156.1845          3.335330          87.87559          87.87559
## null            null 4682.731           0.0000          0.000000          NA          0.00000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions", "NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                               "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents
```

```
##          Nagelkerke
```

```
## CumErr 0.91008791
## CumPres 0.08991209
```

```
sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_sse_results_table.csv"),row.names = TRUE)
sse_table
```

```
##           model p_accounted_for model_deviance diff_CumErr diff_CumErr+CumPres
## 1           preserved ~ CumErr      0.6949104      4526.546  0.00000000      -0.01734607
```

model	p_accounted_for	model_deviance
preserved ~ CumErr	0.6949104	4526.546
preserved ~ CumErr+CumPres	0.7122565	4504.997

model	diff_CumErr	diff_CumErr+CumPres
preserved ~ CumErr	0.0000000	-0.0173461
preserved ~ CumErr+CumPres	0.0173461	0.0000000

```
## 2 preserved ~ CumErr+CumPres      0.7122565      4504.997  0.01734607      0.00000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                                     CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```